MECHATRONICS AND IOT

ASSIGNMENT REPORT – 4

TITLE:

Design and simulate a Smart Refrigerator Monitoring System using DHT11 Sensor. The system monitors internal temperature and humidity conditions to improve food preservation and energy efficiency.

TEAM MEMBERS:

HARISANKARAN S
(24MER013)
MUTHUKUMAR M
(24MER034)

MONESHWARAN C

(24MER032)

PROJECT OVERVIEW

A refrigerator is an important household appliance used to preserve food by maintaining suitable temperature and humidity conditions. Improper temperature or humidity can reduce food quality, increase spoilage, and cause unnecessary energy consumption.

The proposed Smart Refrigerator Monitoring System continuously monitors the internal temperature and humidity using a DHT11 sensor connected to an ESP32 microcontroller. The ESP32 reads the temperature and humidity values and classifies the refrigerator condition as NORMAL, WARNING, or CRITICAL.

The measured temperature, humidity, and system status can be transmitted through Wi-Fi to Adafruit IO Cloud. A cloud dashboard provides remote monitoring and graphical visualization of refrigerator conditions.

The system also provides local warning through a green LED, red LED, OLED display, and buzzer.

PROBLEM STATEMENT

Major problems include:

- Lack of continuous monitoring of refrigerator temperature.
- Difficulty in detecting abnormal temperature conditions.
- Excessive temperature variations may affect food preservation.
- High humidity may affect food quality.
- No automatic local warning system.
- Difficulty in remotely monitoring refrigerator conditions.
- Lack of historical temperature and humidity data.
- Delayed response to abnormal refrigerator conditions.
- Unnecessary energy consumption due to improper operating conditions.

PROPOSED SOLUTION

- Measures internal temperature using a DHT11 sensor.
- Measures internal humidity using the same DHT11 sensor.
- Processes the sensor readings using an ESP32.
- Calculates and evaluates temperature and humidity conditions.
- Classifies the refrigerator condition as **NORMAL, WARNING, or CRITICAL**.
- Displays temperature, humidity, and status on an OLED.
- Controls a green LED during normal conditions.
- Controls a red LED during abnormal conditions.
- Activates a buzzer during critical conditions.
- Connects to Wi-Fi.
- Uploads sensor data to Adafruit IO.
- Provides a cloud dashboard for remote monitoring.
- Stores historical temperature and humidity readings.

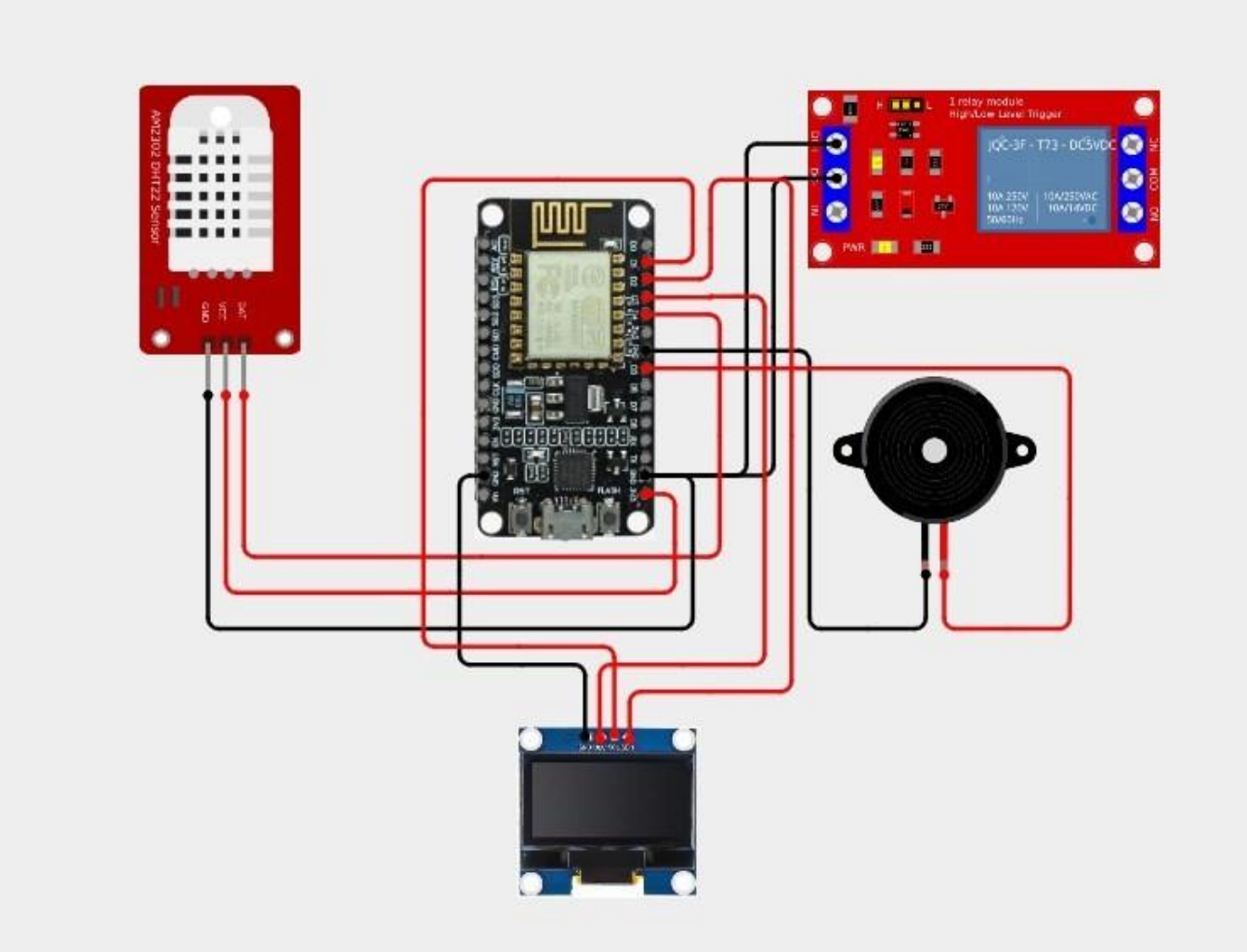
WORKING PRINCIPLE

1. The ESP32 is powered on.
2. The DHT11 sensor, OLED, LEDs, and buzzer are initialized.
3. The ESP32 connects to the Wi-Fi network.
4. The ESP32 connects to Adafruit IO Cloud.
5. The DHT11 sensor measures the internal temperature.
6. The DHT11 sensor measures the internal humidity.
7. The ESP32 receives the sensor readings.
8. The temperature and humidity values are processed.
9. The values are compared with predefined threshold levels.
10. The system determines **NORMAL, WARNING, or CRITICAL** status.
11. The OLED displays temperature, humidity, and status.
12. The green LED indicates normal conditions.
13. The red LED indicates abnormal conditions.
14. The buzzer activates during critical conditions.
15. Temperature and humidity data are uploaded to Adafruit IO.
16. The cloud dashboard displays the data.
17. The process repeats continuously.

CIRCUIT TABLE

Component	Pin	ESP32 GPIO
DHT11 DATA	GPIO 4	Temperature & humidity measurement
OLED SDA	GPIO 21	I2C data
OLED SCL	GPIO 22	I2C clock
Green LED	GPIO 23	Normal indication
Red LED	GPIO 19	Warning/Critical indication
Buzzer	GPIO 25	Alarm
DHT11 VCC	3.2V	Power
DHT11 GND	GND	Groud
OLED VCC	3.3V	Power
OLED GND	GND	Ground

CIRCUIT DESIGN



ALGORITHM

START

↓

Initialize ESP32

↓

Initialize DHT11 Sensor

↓

Initialize OLED

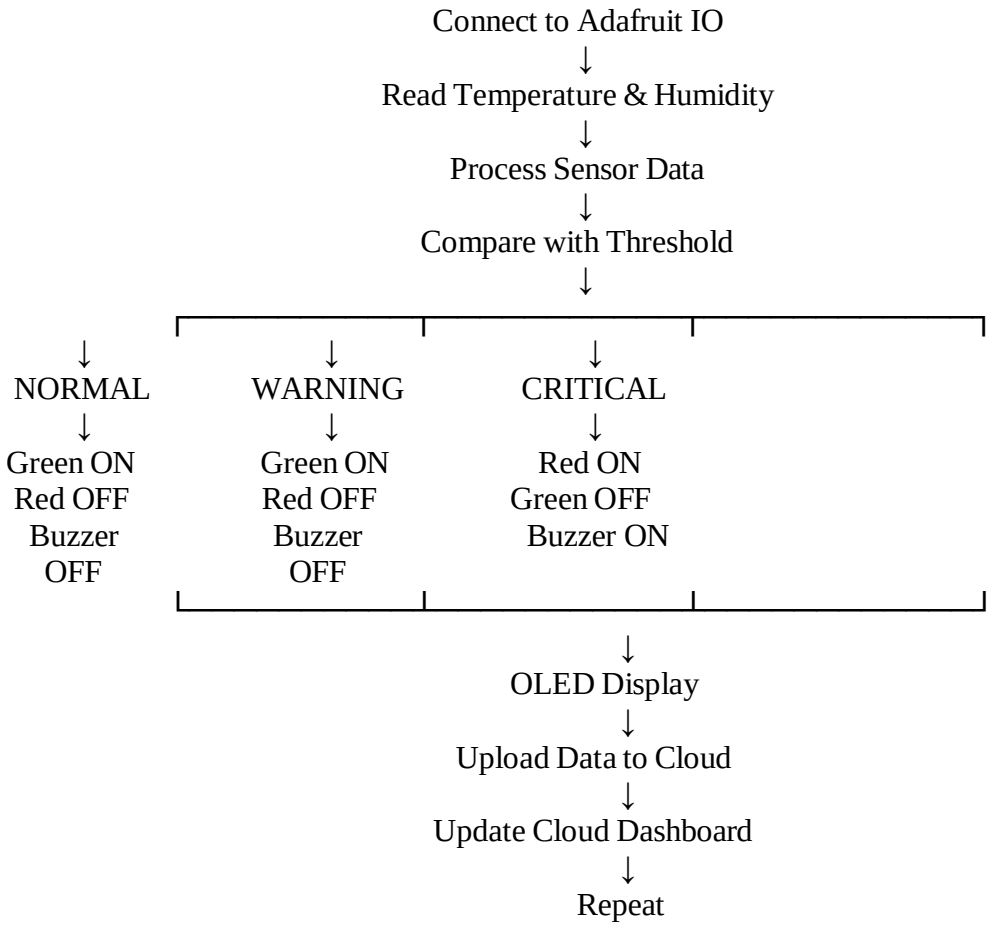
↓

Initialize LEDs and Buzzer

↓

Connect to Wi-Fi

↓



CODING:

```
#include <WiFi.h>
#include <Wire.h>

#include <Adafruit_MQTT.h>
#include <Adafruit_MQTT_Client.h>

#include <DHT.h>

#include <hd44780.h>
#include <hd44780ioClass/hd44780_I2Cexp.h>

// ================================================
// WIFI DETAILS
// ================================================

#define WIFI_SSID "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD "qkts5737"

// ================================================
```

```
// ADAFRUIT IO DETAILS
// ================================================

#define AIO_SERVER "io.adafruit.com"
#define AIO_SERVERPORT 1883

#define AIO_USERNAME "Hair_13"
#define AIO_KEY "aio_EMRC40vci7DQ5Amz5kvOKh6WKHI0"

// ================================================
// DHT11
// ================================================

#define DHTPIN 15
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// ================================================
// I2C LCD
// ================================================

hd44780_I2Cexp lcd;

#define SDA_PIN 21
#define SCL_PIN 22

// ================================================
// RELAY
// ================================================

#define RELAY_PIN 26

// Most common relay modules are ACTIVE LOW
#define RELAY_ON LOW
#define RELAY_OFF HIGH

// ================================================
// BUZZER
// ================================================

#define BUZZER_PIN 25

// ================================================
// MQTT
// ================================================
```

```
WiFiClient client;

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// ====================================================
// ADAFRUIT IO FEEDS
// ====================================================

// Temperature feed
Adafruit_MQTT_Publish temperatureFeed =
Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/hvac-temperature"
  );

// Humidity feed
Adafruit_MQTT_Publish humidityFeed =
Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/hvac-humidity"
  );

// Status feed
Adafruit_MQTT_Publish statusFeed =
Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/hvac-status"
  );

// Buzzer feed
Adafruit_MQTT_Publish buzzerFeed =
Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/hvac-buzzer"
  );

// ====================================================
// CONNECT TO WIFI
// ====================================================
```

```
void connectWiFi()
{
  Serial.println();
  Serial.println("Connecting to WiFi...");

  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("WiFi Connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
}

// ====================================================
// CONNECT TO ADAFRUIT IO
// ====================================================

bool connectMQTT()
{
  if (mqtt.connected())
  {
    return true;
  }

  Serial.println();
  Serial.println("Connecting to Adafruit IO...");

  int8_t ret = mqtt.connect();

  if (ret != 0)
  {
    Serial.print("MQTT Connection Failed: ");
    Serial.println(mqtt.connectErrorString(ret));

    mqtt.disconnect();

    return false;
  }
```

```
  Serial.println("Adafruit IO Connected!");

  return true;
}

// =======================================================
// SETUP
// =======================================================

void setup()
{
  Serial.begin(9600);

  // ---------------------------------
  // DHT11
  // ---------------------------------

  dht.begin();

  // ---------------------------------
  // I2C LCD
  // ---------------------------------

  Wire.begin(SDA_PIN, SCL_PIN);

  int lcdStatus = lcd.begin(16, 2);

  lcd.backlight();
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("HVAC MONITOR");

  lcd.setCursor(0, 1);
  lcd.print("Starting...");

  // ---------------------------------
  // RELAY
  // ---------------------------------

  pinMode(RELAY_PIN, OUTPUT);

  digitalWrite(RELAY_PIN, RELAY_OFF);

  // ---------------------------------
  // BUZZER
```

```
  // --------------------------------

  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);

  delay(2000);

  // --------------------------------
  // WIFI
  // --------------------------------

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("WiFi Connecting");

  connectWiFi();

  // --------------------------------
  // MQTT
  // --------------------------------

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("Adafruit IO");

  lcd.setCursor(0, 1);
  lcd.print("Connecting...");

  connectMQTT();

  delay(2000);

  lcd.clear();
}

// ================================================
// LOOP
// ================================================

void loop()
{
  // ================================================
  // CHECK WIFI
```

```
// ================================================

if (WiFi.status() != WL_CONNECTED)
{
  Serial.println("WiFi disconnected!");

  connectWiFi();
}

// ================================================
// CHECK MQTT
// ================================================

if (!connectMQTT())
{
  Serial.println("MQTT not connected.");

  delay(5000);

  return;
}

// ================================================
// READ DHT11
// ================================================

float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

// ================================================
// SENSOR ERROR
// ================================================

if (isnan(temperature) || isnan(humidity))
{
  Serial.println("DHT11 ERROR!");

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("DHT11 ERROR");

  lcd.setCursor(0, 1);
  lcd.print("Check Wiring");

  digitalWrite(RELAY_PIN, RELAY_OFF);
```

```
  digitalWrite(BUZZER_PIN, LOW);

  delay(2000);

  return;
}

// ================================================
// HVAC STATUS
// ================================================

String hvacStatus;
String buzzerStatus;

if (temperature > 30.0)
{
  // High temperature
  digitalWrite(RELAY_PIN, RELAY_ON);
  digitalWrite(BUZZER_PIN, HIGH);

  hvacStatus = "HIGH TEMPERATURE";
  buzzerStatus = "ON";
}
else
{
  // Normal temperature
  digitalWrite(RELAY_PIN, RELAY_OFF);
  digitalWrite(BUZZER_PIN, LOW);

  hvacStatus = "NORMAL";
  buzzerStatus = "OFF";
}

// ================================================
// LCD DISPLAY
// ================================================

lcd.clear();

lcd.setCursor(0, 0);
lcd.print("Temp:");
lcd.print(temperature, 1);
lcd.write(223);
lcd.print("C");

lcd.setCursor(0, 1);
```

```
lcd.print("Hum:");
lcd.print(humidity, 1);
lcd.print("%");

// ================================================
// SERIAL MONITOR
// ================================================

Serial.println();
Serial.println("==============================");

Serial.print("Temperature : ");
Serial.print(temperature, 1);
Serial.println(" C");

Serial.print("Humidity : ");
Serial.print(humidity, 1);
Serial.println(" %");

Serial.print("HVAC Status : ");
Serial.println(hvacStatus);

Serial.print("Relay : ");

if (temperature > 30.0)
Serial.println("ON");
else
  Serial.println("OFF");

Serial.print("Buzzer : ");
Serial.println(buzzerStatus);

// ================================================
// SEND TEMPERATURE
// ================================================

if (temperatureFeed.publish(temperature))
{
  Serial.println("Temperature -> Adafruit IO : OK");
}
else
{
  Serial.println("Temperature -> Adafruit IO : FAILED");
}

// ================================================
```

```
  // SEND HUMIDITY
  // ================================================

  if (humidityFeed.publish(humidity))
  {
    Serial.println("Humidity -> Adafruit IO : OK");
  }
  else
  {
    Serial.println("Humidity -> Adafruit IO : FAILED");
  }

  // ================================================
  // SEND HVAC STATUS
  // ================================================

  if (statusFeed.publish(hvacStatus.c_str()))
  {
    Serial.println("Status -> Adafruit IO : OK");
  }
  else
  {
    Serial.println("Status -> Adafruit IO : FAILED");
  }

  // ================================================
  // SEND BUZZER STATUS
  // ================================================

  if (buzzerFeed.publish(buzzerStatus.c_str()))
  {
    Serial.println("Buzzer -> Adafruit IO : OK");
  }
  else
  {
    Serial.println("Buzzer -> Adafruit IO : FAILED");
  }

  Serial.println("==============================");

  // ================================================
  // WAIT 5 SECONDS
  // ================================================

  delay(5000);
}
```

SYSTEM

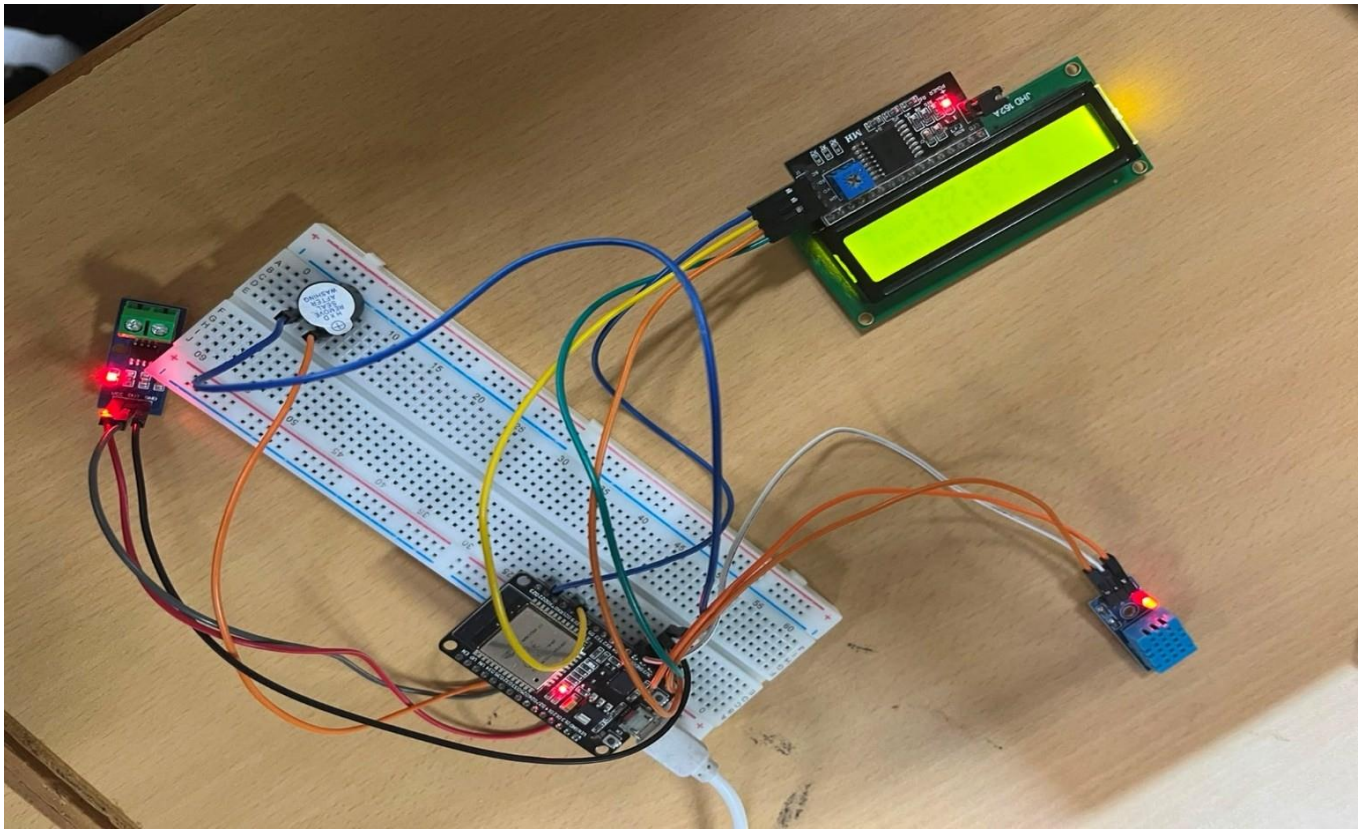
- Start.
- Initialize ESP32.
- Initialize DHT11 sensor.
- Initialize OLED display.
- Initialize LEDs and buzzer.
- Connect ESP32 to Wi-Fi.
- Connect to Adafruit IO.
- Read temperature.
- Read humidity.
- Check whether sensor data is valid.
- Compare temperature and humidity with predefined thresholds.
- Determine NORMAL, WARNING, or CRITICAL condition.
- Control LEDs and buzzer.
- Display readings on OLED.
- Upload readings to Adafruit IO.
- Update cloud dashboard.

BILL OF MATERIALS

S.No.	**Condition**	Temperature	Humidity	Green LED	Red LED	**Buzzer**
1	NORMAL	Suitable range	Suitable range	ON	OFF	OFF
2	WARNING	Slightly abnormal	Slightly abnormal	ON	OFF	OFF
3	CRITICAL	Highly abnormal	Highly abnormal	OFF	ON	ON

PROTOTYPE IMPLEMENTATION

The prototype is built using an ESP32 on a breadboard. The DHT11 sensor is positioned inside a simulated refrigerator environment to measure temperature and humidity. The OLED displays the current readings and system status.
Two LEDs provide visual status indication, while the buzzer provides an audible warning during critical conditions.
The ESP32 connects to Wi-Fi and transmits temperature and humidity data to Adafruit IO Cloud. The cloud dashboard allows the user to remotely monitor refrigerator conditions and observe historical trends.



PERFORMANCE

- Real-time temperature monitoring.
- Real-time humidity monitoring.
- DHT11-based sensing.
- Local OLED display.
- Green/red visual indication.
- Audible critical-condition alarm.
- Wi-Fi connectivity.
- Adafruit IO cloud monitoring.
- Historical temperature and humidity visualization.
- Automatic abnormal-condition detection.
- Remote monitoring capability.
- Low-cost prototype implementation.

RESULTS & PERFORMANCE ANALYSIS